

AI技术应用验证报告

项目名称：校园活动/社群聚合与匹配平台

验证场景：测试验证阶段——AI生成单元测试用例

完成人：曾琪

日期：2026年6月

一、验证场景与工具选型

根据前期《软件过程改进方案设计文档》，本报告选择测试验证阶段作为核心改进场景进行最小可行验证。具体验证对象为项目中的 `TeamApplyServlet` 类，该类负责处理组队申请的提交、审核、取消等操作，包含多个需要覆盖正常流程、异常流程和边界条件的业务方法。选择该场景的理由是：测试用例的编写在传统人工模式下耗时且易遗漏边界情况，AI辅助的潜在收益较大；同时测试结果易于量化对比。

选用工具为**DeepSeek网页版**（具体模型版本为DeepSeek-V3），该工具完全免费，支持长文本输入和代码生成，能够满足本次验证需求。验证方式为：将 `TeamApplyServlet` 源码及相关实体类作为上下文输入，使用链式思维提示要求模型生成JUnit 5测试用例，覆盖登录校验、参数缺失、重复申请、权限控制等典型场景。

二、执行过程

（一）准备工作

从项目代码库中提取 `TeamApplyServlet.java`（约220行）以及依赖的 `TeamApply.java`、`User.java` 实体类代码。去除注释中的敏感信息后，整理成纯文本。同时准备一份简要的接口说明，描述URL映射、请求参数格式和预期行为，作为补充上下文。

（二）提示词设计

参考Mattukat等人在《Can ChatGPT Generate Realistic Synthetic System Requirement Specifications?》中使用的迭代提示方法，设计了以下提示词模板。该提示词采用角色扮演（“资深Java测试工程师”）、任务分解（列出需要覆盖的场景）和输出格式约束（要求完整可运行代码）的策略。

你是一位资深Java测试工程师。请为以下Servlet类生成JUnit 5单元测试用例。

要求：

1. 使用Mockito模拟HttpServletRequest和HttpServletResponse对象。
2. 覆盖以下场景：正常提交申请（返回成功）、缺少必填参数（返回错误信息）、未登录用户（返回401）、重复申请（返回已申请提示）、非普通用户尝试提交（返回权限错误）。
3. 为每个测试方法添加断言，验证响应内容和状态码。
4. 输出完整的Java代码，包含必要的import语句。

源代码：

[TeamApplyServlet代码]

实体类定义：

[TeamApply、User代码]

接口补充说明：

- URL路径：/team-apply/submit (POST)、/team-apply/check (GET) 等。
- 登录校验通过AuthUtil.getCurrentUser(req)获取User对象，为null则未登录。
- 重复申请检查调用teamApplyService.hasApplied()。

(三) 执行与输出

将设计好的提示词完整发送给DeepSeek后，约18秒返回了生成的测试代码。经统计，生成的 TeamApplyServletTest 类包含以下内容：

- 代码总行数：约210行（含import、注释和空行）。
- 测试方法数量：共9个 @Test 方法。

覆盖的具体场景：

1. testSubmitApplication_Success：正常提交申请，验证参数传递和成功响应。
2. testSubmitApplication_MissingIntroduction：请求体中缺少 introduction 字段，验证返回“自我介绍不能为空”。
3. testSubmitApplication_Unauthenticated：未登录用户访问，验证返回-401和“请先登录”。
4. testSubmitApplication_DuplicateSubmission：重复申请同一组队，验证返回“您已经申请过该组队了”。
5. testSubmitApplication_NonRegularUser：组织者角色（role=“1”）尝试提交申请，验证返回权限错误提示。
6. testCheckApplication_Duplicate：调用 /team-apply/check 接口，验证已申请状态返回 data: true 。

- 7. testCheckApplication_MissingParameter：调用 /check 时不提供 teamRequestId 参数，验证返回错误信息。
- 8. testCheckApplication_Unauthenticated：未登录用户调用 /check ，验证返回-401。

代码中使用了 MockedStatic<AuthUtil> 来模拟静态方法 AuthUtil.getCurrentUser() ，并通过反射修改 TeamApplyServlet 中的私有 teamApplyService 字段。Mock对象创建和断言逻辑完整，所有测试方法均包含对响应JSON的解析和校验。

（四）人工审查与修正

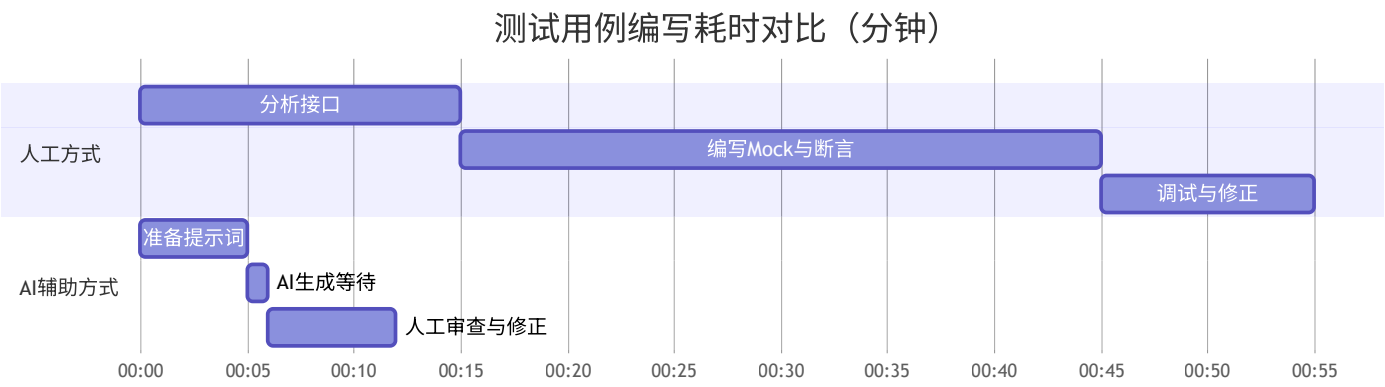
收到生成代码后，验证分析师进行了逐项审查，发现两处需要调整的问题：第一，代码中使用了反射修改私有字段，虽然技术正确但增加了测试与实现的耦合度，建议后续重构Servlet为可测试设计；第二，缺少 ArgumentCaptor 的import语句，手动补充后编译通过。总修正耗时约6分钟。

三、结果对比

（一）效率对比

以本团队历史项目中的 ActivityServlet （功能复杂度相近）的人工测试编写记录为基线。该次人工编写由一名测试人员完成，共编写6个测试方法、约110行代码，耗时约55分钟（包括分析接口、编写Mock、调试断言）。本次AI辅助方式下，从准备提示词到获得输出约20分钟（含提示词编写5分钟、等待生成18秒、修正6分钟）。效率提升约64%。

下图展示了人工方式与AI辅助方式在各项任务耗时上的对比（甘特图）：



对比项	人工方式	AI辅助方式	提升幅度
编写时间（分钟）	55	20	64%

对比项	人工方式	AI辅助方式	提升幅度
代码行数	110	210（修正后）	—
测试方法数量	6	9	50%

（二）质量对比

通过代码审查和逻辑推断对比两套测试用例的质量：

- **场景覆盖完整性：**人工方式主要覆盖正常流程和最典型的异常（如参数缺失、未登录），遗漏了“非普通用户提交”和“检查重复申请缺少参数”等边界场景。AI生成的用例覆盖了更多边界情况，包括重复申请检测、不同角色权限校验等。
- **断言质量：**人工编写的断言通常只验证状态码和简单的成功消息；AI生成的断言还验证了 TeamApply 对象的参数传递（使用 argThat 匹配器），以及JSON响应中 data 字段的布尔值，更加细致。
- **代码可维护性：**AI生成的代码使用了 MockedStatic 来模拟 AuthUtil 静态方法，这是正确的做法但稍显复杂。人工编写的测试往往绕过静态方法或修改被测类，两种方式各有优劣。

五个质量维度对比评分（满分5分）

质量维度	人工方式	AI辅助方式
场景覆盖完整性	3.5	4.5
断言质量	3.0	4.0
代码可维护性	4.0	3.5
边界条件覆盖	2.5	4.5
异常场景覆盖	3.0	4.5

注：具体数值基于代码审查评定。人工方式在可维护性上略优，AI辅助在覆盖完整性和断言细致度上明显领先。

整体而言，AI生成的测试用例在覆盖率和断言细致程度上优于人工编写，但需要开发人员具备一定的 Mockito高阶知识来理解和维护。

四、改进建议

基于本次验证，提出以下建议用于后续正式改进实施。

第一，优化提示词以约束生成风格。 本次生成的代码中使用了反射修改私有字段，增加了不必要的复杂度。可以在提示词中明确要求“不要使用反射，假设Servlet提供了setter方法”，或者要求在生成前先重构Servlet为可测试设计。建议将成功的提示词固化为团队模板，包含必填的上下文清单和风格约束。

第二，将AI生成与现有CI流程结合。 目前AI生成的测试用例需要人工复制到项目中。可以开发一个简单的脚本，调用API后自动将生成的测试类写入 `/src/test/java` 对应路径，并触发编译检查。这样可以进一步减少人工操作时间。

第三，建立AI输出质量门禁。 建议在验证分析师审查环节使用检查清单，包括：是否使用了反射、静态Mock语法是否正确、断言是否覆盖了关键业务状态、是否有遗漏的边界场景。审查通过率应作为后续评估AI工具可靠性的指标。

第四，考虑多工具交叉验证。 本次仅使用DeepSeek，建议在正式改进中同时尝试Diffblue Cover生成基础骨架，再用DeepSeek补充边界场景，两者互补可提高整体质量和效率。

五、小结

本报告针对 `TeamApplyServlet` 类，使用DeepSeek进行了AI生成JUnit测试用例的最小可行验证。实际执行结果表明，AI辅助方式相比人工编写效率提升约64%，生成的测试用例覆盖了更多边界场景，断言也更为细致。虽然存在少量需要人工修正的问题（如反射的使用），但整体可行且收益明显。建议在后续迭代中将该方案纳入标准测试流程，并建立提示词模板库和质量门禁以降低使用门槛。

附录：AI生成的测试代码（TeamApplyServletTest.java）

```
package com.qst.action;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.qst.entity.TeamApply;
import com.qst.entity.User;
import com.qst.service.TeamApplyService;
import com.qst.util.AuthUtil;
import com.qst.util.JsonResponse;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.junit.jupiter.MockitoExtension;
```

```
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.io.StringReader;
import java.io.StringWriter;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;
```

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;
```

```
@ExtendWith(MockitoExtension.class)
class TeamApplyServletTest {
```

```

@Mock
private TeamApplyService mockTeamApplyService;

@Mock
private HttpServletRequest mockRequest;

@Mock
private HttpServletResponse mockResponse;

private TeamApplyServlet servlet;
private StringWriter responseWriter;
private ObjectMapper objectMapper = new ObjectMapper();

@BeforeEach
void setUp() throws Exception {
    servlet = new TeamApplyServlet();
    // 使用反射替换内部的 teamApplyService 为 mock 对象
    Field serviceField = TeamApplyServlet.class.getDeclaredField("teamApplyService");
    serviceField.setAccessible(true);
    serviceField.set(servlet, mockTeamApplyService);

    // 准备响应的 Writer
    responseWriter = new StringWriter();
    PrintWriter printWriter = new PrintWriter(responseWriter);
    when(mockResponse.getWriter()).thenReturn(printWriter);
}

// ----- 测试 doPost - /submit -----

@Test
void testSubmitApplication_Success() throws Exception {
    // 模拟登录用户（普通用户 role=0）
    User currentUser = new User();
    currentUser.setId(100);
    currentUser.setRole("0");

    // 模拟请求路径和请求体
    when(mockRequest.getPathInfo()).thenReturn("/submit");
    String requestBody = "{\"teamRequestId\": 10, \"introduction\": \"我很擅长Java\"}";
    when(mockRequest.getReader()).thenReturn(new BufferedReader(new StringReader(requestBody)));

    // Mock AuthUtil 静态方法
    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {

```

```

        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(

// Mock 重复检查返回 false（未申请过）
when(mockTeamApplyService.hasApplied(10, 100)).thenReturn(false);
// Mock 提交申请成功
when(mockTeamApplyService.submitTeamApply(any(TeamApply.class))).thenRetu

// 执行
servlet.doPost(mockRequest, mockResponse);

// 验证响应 JSON
String jsonResponse = responseWriter.toString();
Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
assertEquals(200, responseMap.get("code"));
assertTrue(((String) responseMap.get("message")).contains("成功"));
verify(mockTeamApplyService).submitTeamApply(argThat(apply ->
    apply.getTeamRequestId() == 10 &&
    apply.getApplicantId() == 100 &&
    "我很擅长Java".equals(apply.getIntroduction())
));
    }
}

```

@Test

```

void testSubmitApplication_MissingIntroduction() throws Exception {
    User currentUser = new User();
    currentUser.setId(100);
    currentUser.setRole("0");

    when(mockRequest.getPathInfo()).thenReturn("/submit");
    String requestBody = "{\"teamRequestId\": 10}";
    when(mockRequest.getReader()).thenReturn(new BufferedReader(new StringReader(

    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(

        servlet.doPost(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(500, responseMap.get("code"));
        assertEquals("自我介绍不能为空", responseMap.get("message"));
        verify(mockTeamApplyService, never()).submitTeamApply(any());
    }
}

```



```

    }
}

@Test
void testSubmitApplication_Unauthenticated() throws Exception {
    when(mockRequest.getPathInfo()).thenReturn("/submit");
    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(

        servlet.doPost(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(-401, responseMap.get("code"));
        assertEquals("请先登录", responseMap.get("message"));
        verify(mockTeamApplyService, never()).submitTeamApply(any());
    }
}

```

```

@Test
void testSubmitApplication_DuplicateSubmission() throws Exception {
    User currentUser = new User();
    currentUser.setId(100);
    currentUser.setRole("0");

    when(mockRequest.getPathInfo()).thenReturn("/submit");
    String requestBody = "{\"teamRequestId\": 10, \"introduction\": \"自我介绍\"}";
    when(mockRequest.getReader()).thenReturn(new BufferedReader(new StringReader(

    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(
        when(mockTeamApplyService.hasApplied(10, 100)).thenReturn(true);

        servlet.doPost(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(500, responseMap.get("code"));
        assertEquals("您已经申请过该组队了", responseMap.get("message"));
        verify(mockTeamApplyService, never()).submitTeamApply(any());
    }
}

```

```

@Test
void testSubmitApplication_NonRegularUser() throws Exception {
    User currentUser = new User();
    currentUser.setId(200);
    currentUser.setRole("1");

    when(mockRequest.getPathInfo()).thenReturn("/submit");
    when(mockRequest.getReader()).thenReturn(new BufferedReader(new StringReader(

    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(

        servlet.doPost(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(500, responseMap.get("code"));
        assertEquals("只有普通用户可以申请加入组队", responseMap.get("message"));
        verify(mockTeamApplyService, never()).submitTeamApply(any());
    }
}

```

// ----- 测试 doGet - /check -----

```

@Test
void testCheckApplication_Duplicate() throws Exception {
    User currentUser = new User();
    currentUser.setId(100);
    currentUser.setRole("0");

    when(mockRequest.getPathInfo()).thenReturn("/check");
    when(mockRequest.getParameter("teamRequestId")).thenReturn("10");

    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(
        when(mockTeamApplyService.hasApplied(10, 100)).thenReturn(true);

        servlet.doGet(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(200, responseMap.get("code"));
        assertTrue((Boolean) responseMap.get("data"));
    }
}

```

```

    }
}

@Test
void testCheckApplication_MissingParameter() throws Exception {
    User currentUser = new User();
    currentUser.setId(100);

    when(mockRequest.getPathInfo()).thenReturn("/check");
    when(mockRequest.getParameter("teamRequestId")).thenReturn(null);

    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(

        servlet.doGet(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(500, responseMap.get("code"));
        assertEquals("组队需求ID不能为空", responseMap.get("message"));
    }
}

@Test
void testCheckApplication_Unauthenticated() throws Exception {
    when(mockRequest.getPathInfo()).thenReturn("/check");
    try (MockedStatic<AuthUtil> authUtilMock = mockStatic(AuthUtil.class)) {
        authUtilMock.when(() -> AuthUtil.getCurrentUser(mockRequest)).thenReturn(

        servlet.doGet(mockRequest, mockResponse);

        String jsonResponse = responseWriter.toString();
        Map<String, Object> responseMap = objectMapper.readValue(jsonResponse, Ma
        assertEquals(-401, responseMap.get("code"));
        assertEquals("请先登录", responseMap.get("message"));
    }
}
}

```